Activity 0.1 – Everyday Task as an Algorithm

Title: From Daily Routine to Algorithm

Instructions to Students

1. Choose one simple daily activity (examples: making tea, preparing to go to class, booking a cab).
2. Write the steps in clear, numbered points.
3. Make sure each step is:
   - Unambiguous
   - In correct order
   - Does not skip important details

Template

1. Activity chosen: Preparing to go to class
2. Write the steps:
   1. Finish lunch
   2. Pack the bag with requird books
   3. Carry I'd card
   4. Check the class timings
   5. Leave hostel on time
   6. Reach the classroom on time
3. Check your algorithm:
   - Can a friend follow it without asking questions? Yes / No ✓
   - If "No", refine the steps here:
     - _____

Activity 0.2 – From Steps to Pseudo-code

Title: Writing Pseudo-code with IF and REPEAT

Instructions

1. Take your steps from Activity 0.1.
2. Identify where decisions happen (IF/ELSE).
3. Identify where repetition happens (REPEAT / WHILE).
4. Rewrite your algorithm using the following keywords:
   - IF, ELSE, ELSE IF
   - REPEAT, WHILE, UNTIL

Template

1. Original activity: Preparing to go to class
2. Pseudo-code version:

text

START

Finish lunch, Pack bags, Carry I'd card

IF Class timing is updated THEN

go to class

ELSE

wait for the time to be updated

ENDIF

REPEAT

Check time

UNTIL it is time to leave the hostel, to reach the class on time

END

3. Reflection:
   - Where did you use IF/ELSE? to check the class timings
   - Where did you use REPEAT/WHILE? check time to leave hostel

Activity 1.1 – Run and Edit Given Code

Title: First Python Script – Edit and Run

Instructions

1. Your teacher will give you a small Python script (example below).
2. Do NOT write new code first; only change text or numbers.
3. Run the program and observe the output.

Example Script

python

```
name = "Student"
age = 18
print("Hello,", name)
print("You are", age, "years old.")
```

Tasks

1. Change name and age to your own values.
   - name: "Achintya"
   - age: 21
2. Run the program. What is the output?

text

Output:

"Hello", Achintya
You are 21 years old

3. Change the printed message to:
   - "Welcome to Python, <your name>"

Write your edited code here:

python

```
name = "Achintya"
age = 21
print ("Welcome to Python,", name)
```

Edited code:

Output : Welcome to Python, Achintya

4. Reflection:
   - What changed in the output and why?

As values and message were modified the output changed.

Activity 1.2 – Predict–Then–Run

Title: Predict Program Output

Instructions

1. Look at each code snippet.
2. BEFORE running, write what you think the output will be.
3. Then run it and compare.

Snippet A

python

```
x = 5
y = 2
z = x + y * 3
print(z)
```

- Predicted output: 11
- Actual output: 11

Snippet B

python

```
a = 10
b = a - 3
a = a + 1
print(a, b)
```

- Predicted output: 11, 7
- Actual output: 11, 7

Reflection

- Where was your prediction wrong? Why?

The prediction was right

Activity 2.1 – Calculator in Steps

Title: From Word Problem to Code

Problem 1

A student has marks in 3 subjects. Find total and average.

1. Write the steps in plain language:
   1. Enter marks of 3 subjects
   2. Add all marks
   3. Calculate average by dividing the total with 3
   4. Print total and average
2. Now convert to Python (fill in the blanks):

python

mark1 = 80

mark2 = 75

mark3 = 70

total = mark 1 + mark2 + mark 3

average = total / 3

print("Total =", total)

print("Average =", average)

3. Test with your own numbers and record output.

Output → Total = 225

Average = 225/3 = 75.0

Activity 2.2 – Fix the Errors

Title: Debugging Simple Python

Instructions

Each snippet has an error.

1. Write what you think is wrong.
2. Correct the code.

Snippet 1

python

print("Hello World)

- Error: Missing double inverted commas
- Corrected code: "Hello World"

python

undefined

Snippet 2

python

a = 10

b = "5"

c = a + b

print(c)

- Error: A string and an int cannot be added
- Corrected code:

```
a = 10
b = int("5")
c = a + b
print(c)
```

python

undefined

Reflection:

- What did you learn about syntax vs logic errors?

Syntax → writing mistake

Logic → thinking mistake.

Activity 3.1 – Rule-based Classification

Title: Grades with IF/ELSE

Rule

- If mark ≥ 90 : Grade A
- Else if mark ≥ 75 : Grade B
- Else : Grade C

Tasks

1. Draw the decision in simple text:
   - Condition 1: mark ≥ 90
   - Condition 2: mark ≥ 75
2. Now write Python code:

python

```
mark = int(input("Enter mark: "))
if mark >=90 :
    print("Grade A")
elif mark >=75 :
    print("Grade B")
else:
    print("Grade C")
```

3. Test with at least 3 marks and note outputs.

mark = int (input (" Enter mark: "))

Output :

Enter mark 85
Output Grade B

Enter mark 95
Grade A

Enter mark 65
Grade C

Activity 3.2 – Mini Problem Bank

Title: Structured Problem Solving with IF/ELSE

For each question, fill the template.

Template

1. Restate the problem in one sentence:

   odd or even numbers

2. Conditions needed (bullets):

   - if n % 2 == 0 : ⇒ even
   - else : odd

3. Python code:

python

```
# Your solution
n = int (input ("Enter the number : "))
if n % 2 == 0 :
    print ("Even")
else :
    print ("odd")
```

4. Test cases and results:

   - Input: 2 Output: even
   - Input: 3 Output: odd

Use this template for:

- Odd or even number.
- Ticket discount if age < 12 or > 60.
- Check if a year is a leap year (simple rule).

Conditions needed for

- Ticket discount
  → if age < 12 or age > 60 - discount
  → else - no discount
- leap years
  → if year % 4 == 0 : - leap year
  else - not a leap year.

Ticket discount

```
age = int (input ("Enter age : "))
if age < 12 or age > 60 :
    print ("Discount applicable")
else :
    print ("No discount")
```

~~leap year~~ Output

Enter age 11
No discount

Enter age 13
Discount applicable

leap year

```
year = int (input ("Enter year :"))
if year % 4 == 0 :
    print ("leap year")
else :
    print ("Not a leap year")
```

Enter year 2022
Not a leap year

Enter year 2020
leap year.

Activity 4.1 – Sum with Loops

Title: Sum from 1 to N

Steps

1. In plain language:
   - What is "sum from 1 to N"? n(n+1)/2
2. Write the algorithm:

1. Read the value of N
2. Set total to 0
3. Use loop, for
4. Print the sum

3. Now code it:

python

```
N = int(input("Enter N: "))
total = 0
for i in range( 1, (N+1) ):
  total = total + i
print("Sum =", total)
```

4. Try N = 5, 10, 100 and check results.

Enter N : 5 10 100

Sum = 15 55 5050

Activity 5.1 – Working with Lists

Title: Analyze Marks List

Given

python

marks = [56, 78, 90, 45, 88]

1. In words, write how to find:
   - Maximum mark: highest value
   - Average mark: sum / count [5] - marks
   - Count of marks ≥ 60: check each value
2. Now write Python:

python

```
marks = [56, 78, 90, 45, 88]


# Max
max_mark = max (marks)


# Average
total = 0
for m in marks:
  total = total + m
average = total / len(marks)


# Count >= 60
count = 0
for m in marks:
  if m >= 60:
    count = count + 1


print("Max:", max_mark)
print("Average:", average)
print("Count >= 60:", count)
```

Activity 5.2 – Choosing an Approach

Title: Which Method is Better?

Problem

Given a list of 1000 numbers, find how many are positive.

1. Approach A (loop) – describe in words:

In loop, It goes through each number in the list one by one

2. Approach B (using built-in functions or list comprehension) – describe:

In built-in functions or list comprehension, create a new list which only as positive number using condition.

3. Which is easier to understand for you now? Why?

Approach A is easier, because it goes through the list one by one

4. Implement one approach in Python:

python

numbers = [10, -3, 5, 0, 8, -2] *# example*

 # Your code

```
numbers = [10, -3, 5, 0, 8, -2]
count = 0

for n in numbers
    if n > 0:
        count = count + 1

print ("Positive numbers count =", count)
```

Activity 6.1 – Creating a Function

Title: Wrap Logic into a Function

Task

Turn average-of-marks logic into a function.

1. In words: what does the function do?

Function calculates and returns the average of list of marks

2. Write code:

python

```
def average_marks(marks_list):
    total = 0
    for m in marks_list:
        total = total + len(marks_list)
    avg = total / len(marks_list)
    return avg

data = [60, 70, 80]
print("Average =", average_marks(data))
```

3. Test with another list and note output.

average = 70.0.

Activity 6.2 – Problem-Solving Template

Title: Standard Template for Any Problem

Use this for any new problem. - Check whether a number is positive, negative or zero

1. Problem understanding

   - What is given (inputs)?

     A number

   - What is required (outputs)?

     Positive / Negative / Zero

   - Any constraints (limits, ranges)?

     Number can be any integer

2. Analysis

   - Write steps in bullets (algorithm):
     - Read the number
     - Check if number > 0
     - Else if number < 0
     - Else print zero

3. Design

   - Will you use loops? Which? Not required
   - Will you use conditions (if/else)? Yes, if - elif - else
   - Will you use functions? Which ones? Not required
   - What data structures (list, etc.)? Not required

4. Code (Python)

python

```
# Your implementation
num = int (input ("Enter a number :"))
if num > 0 :
    print ("Positive number")
elif num < 0 :
    print ("Negative number")
else : print ("zero")
```

5. Test

   - Test case 1: input 19 → output positive
   - Test case 2: input -60 → output Negative
   - Edge case: input 40 → output Positive

```
num = int (input ("Enter a number :"))
if num > 0 :
    print ("Positive number")
elif num < 0 :
    print ("Negative number")
else :
    print ("zero")
```

Activity 7.1 – **Capstone Problem**

Title: Simple Fee Calculator

Problem

A class charges a base fee.

- If student is below 12 years, 50% discount.
- If above 60, 30% discount.
- Otherwise, no discount.

Use the full template from Activity 6.2 here.

(Attach a blank copy of the template and ask students to fill all five sections.)

---

1) Problem Understanding

Inputs : Age of the student, base fee

Outputs : Final fee after discount

Constraints : Age and fees cannot negative

2) Analysis

- Read age of student
- Set base fee
- Check if age is below 12
- Elif age is above 60
- Apply discount accordingly
- Display final fee

3) Design

- Loops - not required
- Conditions - if - elif - else
- Functions - not required
- Data structures - not required

4) Code

```
fee = 2000
age = int(input("Enter age :"))

if age < 12 :
    fee = fee * 0.5
elif age > 60 :
    fee = fee * 0.7
print("Final fee =", fee)
```

Output

```
Enter age : 54
fee = 2000.
```

Activity 7.2 – Reflection Sheet

Title: How I Solve a New Problem

1. When I see a new problem, my first step is:

   To understand what the problem is asking

2. Before coding, I make sure I know:

   - Inputs: Values given in the problem
   - Outputs: what result is expected

3. The tools I think about (loops, if, functions) are:

   whether to use conditions, loops or functions based on the problem

4. One thing I will do differently from now on:

   I will try writing steps before writing the code.

Activity 4.2 – Star Patterns

Title: Loops for Patterns

Pattern (triangle)

```
*
**
***
****
```

1. Explain the pattern in words:

Each row prints stars equal to row number

2. Write code:

python

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
    print("*" * i)
```

3. Try different rows and observe.

Output

```
enter number of rows : 5
*
* *
* * *
* * * *
* * * * *
```

```
enter number of rows : 3
*
* *
* * *
```

```
rows = int(input("Enter number of rows:"))
for i in range (1, rows + 1):
    for j in range (rows - i):
        print (" ", end = " ")
    for k in range (i):
        print ("*", end = " ")
    print ( )
```

Output :

```
Enter number of rows : 5
            *
          * *
        * * *
      * * * *
    * * * * *
```